

SHORE-RL: Shortening Effective Horizon for Long-Horizon Residual Reinforcement Learning

Supplementary Material

Anonymous submission

Abstract

This supplement documents the SHORE-RL objective and the implementation details currently recoverable from the retained code and artifacts, followed by the simulation and real-robot experimental protocols. The two venues share the same residual policy parameterization and actor-critic update, while instantiating different transition collectors, reward sources, and replay samplers. We additionally document the ablation design, available run provenance, qualitative dynamics-model evidence, and the physical-robot evaluation controls needed to interpret the main-paper results. Unavailable validation results and provenance fields are marked explicitly rather than inferred.

A Method and Implementation Details

The main paper introduces the residual controller, self-derived waypoint, and two dense-reward choices. Here we give the method-level optimization objective and the venue-parameterized update of Algorithm 1, in the notation the two venues reuse. Shared architectural settings and the simulation defaults are collected in Table 2; venue-specific numbers and overrides appear in Section B.3 and Section C.5.

A.1 Residual Coordinates and Decision Time Scale

We write the navigator state as $e_t = \text{Std}([\psi_b(o_t); q_t])$, where ψ_b is the frozen base-policy feature and q_t is proprioception; implementation tables occasionally abbreviate this feature as s_t . The residual actor and critic do *not* use ψ_b : they share a trainable multi-view image encoder, distinct from ψ_b and updated only through the critic loss (Section A.4). Both additionally read proprioception and the waypoint z_t ; the actor also receives the base action a_t^b , whereas the critic conditions on the projected combined action a_t and does not take a_t^b as a separate input.

Let f_θ denote the actor’s pre-squash output. The deterministic scaled residual mean and projected normalized command are

$$\begin{aligned}\mu_\theta(o_t, a_t^b, z_t) &= s_a \tanh(f_\theta(o_t, a_t^b, z_t)), \\ a_t &= \Pi_A(a_t^b + \delta_t),\end{aligned}$$

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

where collection samples δ_t around μ_θ using the exploration rule in Section A.4, while evaluation uses $\delta_t = \mu_\theta$. The same projected normalized action a_t is passed to the collector (and to the dynamics model in imagination) and stored in replay. The environment may subsequently convert this normalized action to the robot’s physical command coordinates, but the critic is never trained on a distinct unprojected combined action.

The residual coordinates satisfy $\mu_\theta \in [-s_a, s_a]^d$. Zero-initializing the actor final layer makes the deterministic scaled mean μ_θ exactly zero, so the initial evaluation policy equals the frozen base. Training rollouts add exploration noise in scaled-action coordinates (Section A.4); the initial behavior policy is therefore the base plus exploration noise.

The residual decision time scale is venue-specific. In the *simulation* venue the residual chunk length is 1, so the residual is produced once per simulation control step. A fresh a_{res} is sampled and added to the current base action at every step rather than held fixed across a chunk. The frozen base replans its action chunk every execution horizon of 10 steps, as detailed in Section B.2. A 30-step variant is also run on LIBERO. These base actions are executed open-loop between replans, so the base is open-loop within a chunk while the residual stays closed-loop per step. In these simulated experiments the episode horizons (Table 3) count simulation control steps, which coincide one-to-one with residual decisions. The *real-robot imagination* venue (Section C) is different: the dynamics-model bridge forces a residual chunk length of 50 (Sections C.1 and C.5), so a single residual decision emits the full 50-step residual chunk—executed open-loop and scored as one chunk-level transition—rather than one closed-loop correction per step, and one RL decision there corresponds to 50 control actions.

A.2 Waypoint Implementation Details

The goal-conditioned value $V_{\text{gc}}(e, g)$ and navigator π_h are pretrained on demonstrations using standardized frozen-base features. For M demonstration terminal features, the fixed

deployment goal is

$$\begin{aligned}\bar{e}_T &= \frac{1}{M} \sum_{j=1}^M e_T^{(j)}, \\ j^* &= \arg \min_j \|e_T^{(j)} - \bar{e}_T\|_2, \\ g_* &= e_T^{(j^*)}.\end{aligned}$$

Thus g_* is an observed terminal feature, not a synthetic average.

Goal bottleneck and twin value heads. The learned goal representation is

$$\begin{aligned}u(e, g) &= \begin{cases} [g; e], & \text{dual-arm,} \\ g, & \text{LIBERO,} \end{cases} \\ \phi(e, g) &= \sqrt{d_z} \frac{\tilde{\phi}(u(e, g))}{\|\tilde{\phi}(u(e, g))\|_2 + \epsilon}, \quad d_z = 10.\end{aligned}$$

The bottleneck ϕ is learned during offline goal-value pre-training and is frozen thereafter. The headline value implementation has two heads. With $i \in \{1, 2\}$, its per-head bootstrap, shared advantage, and loss are

$$\begin{aligned}q_i &= r_g + \gamma m_g \bar{V}_{gc,i}(e_{t+1}, g), \\ A_g &= r_g + \gamma m_g \min_j \bar{V}_{gc,j}(e_{t+1}, g) \\ &\quad - \frac{1}{2} \sum_j \bar{V}_{gc,j}(e_t, g), \\ \mathcal{L}_{gc} &= \sum_i \mathbb{E} \left[w_\tau(A_g) (q_i - V_{gc,i}(e_t, g))^2 \right], \\ w_\tau(A) &= \begin{cases} \tau, & A \geq 0, \\ 1 - \tau, & A < 0. \end{cases}\end{aligned}$$

Here $r_g = \mathbf{1}[e_t = g] - 1$ and $m_g = 1 - \mathbf{1}[e_t = g]$, so reaching the sampled goal yields reward 0 and cuts the bootstrap; every other state yields -1 . This is the compact objective shown in the main paper with its twin-head implementation restored. Goals for value training are sampled as current state (0.2), a same-trajectory future state (0.5), or a uniformly random state (0.3). Only the future branch uses a geometric time offset with parameter $1 - \gamma$.

Navigator training and deployment. The high-level policy is trained by

$$\begin{aligned}A_h &= V_{agg}(e_{w_t}, g) - V_{agg}(e_t, g), \\ w(A_h) &= \min\{\exp(\beta A_h), 100\}, \\ \mathcal{L}_h &= -\mathbb{E}[w(A_h) \log \pi_h(\phi(e_t, e_{w_t}) \mid e_t, g)].\end{aligned}$$

Offline, the clamp-to-goal sampler chooses the waypoint index as the earlier of the sampled goal and k steps ahead; its random-goal probability is 0.3. We use $k = 15$ for dual-arm tasks and $k = 25$ for LIBERO, $\beta = 1$, and the weight cap shown above. Offline training aggregates the two value heads by their mean. The online update instead uses their minimum.

At deployment, the navigator never reads a future observation. Let $\mu_t = \mathbb{E}_{\pi_h(\cdot \mid e_t, g_*)}[z]$. The actual actor condition is

$$z_t = \begin{cases} \mu_t, & \text{dual-arm,} \\ \sqrt{d_z} \mu_t / (\|\mu_t\|_2 + \epsilon), & \text{LIBERO.} \end{cases}$$

Thus radius- $\sqrt{10}$ deployment renormalization is a LIBERO-specific switch, not a universal definition of the waypoint. In all waypoint-enabled simulation runs, including the dual-arm and LIBERO runs, V_{gc} and π_h are updated after 2,000 online transitions, once per residual UTD block, using Adam with learning rate 10^{-5} , batch size 256, and a 50/50 demonstration-online split. Only the two value heads and navigator are updated; the base policy, base encoder, and ϕ remain frozen.

A.3 Potential and Stage-Bonus Implementations

The two dense-reward choices are not the same functional form, and only the second is potential-based (Ng, Harada, and Russell 1999; Devlin and Kudenko 2012). The headline stage-structured simulation configuration uses

$$\tilde{r}_t = r_t + b[G_{t+1} - G_t]_+, \quad b = 1,$$

where G_t is the running maximum of the stage-detector output. Hence it is latched and nondecreasing, and the positive-part operator is only defensive. This reward is a direct transition bonus paid once when a new stage is first reached. It is not a telescoping potential difference, so we make no policy-invariance claim for this branch.

The self-derived branch uses an independently trained single-state progress value V_p . On successful demonstrations,

$$\begin{aligned}r_t^p &= \mathbf{1}[\text{successful demonstration terminal at } t], \\ y_t^p &= r_t^p + \gamma(1 - d_t^p) \bar{V}_p(e_{t+1}), \\ \mathcal{L}_p &= \mathbb{E}[\ell_2^2(y_t^p - V_p(e_t))].\end{aligned}$$

This 1/0 convention is distinct from the 0/-1 goal-reaching reward used by V_{gc} . After offline training, V_p is frozen and defines

$$\Phi(e) = c V_p(e), \quad c = \frac{\max\{K - 1, 1\}}{\max\{V_{\max} - V_{\min}, 10^{-6}\}},$$

where K is the number of task stages and $V_{\max}, V_{\min}$ are measured over offline states. For real-robot tasks without a stage decomposition we set $K = 1$, giving $c = 1/(V_{\max} - V_{\min})$. The privilege-free simulation ablation and imagination training use this frozen potential. The LIBERO and CANSort headline runs use no shaping, so their reward remains r_t . The goal-conditioned value supplies only the waypoint; the separate single-state value supplies only Φ .

Terminal handling differs between true simulator episodes and artificial imagination boundaries. Direct simulator collection uses

$$\begin{aligned}\tilde{r}_t^{\text{sim}} &= r_t + \gamma(1 - d_t^{\text{sim}}) \Phi(e_{t+1}) - \Phi(e_t), \\ d_t^{\text{sim}} &= \text{terminated}_t \vee \text{truncated}_t.\end{aligned}$$

The same flag zeros the critic bootstrap, so this is internally consistent with the finite-horizon objective actually

optimized; no invariance claim is made for the underlying infinite-horizon MDP. In imagination, the segment-end truncation is artificial and must not zero the endpoint potential:

$$\tilde{r}_t^{\text{imag}} = \gamma \Phi(\hat{e}_{t+1}) - \Phi(e_t).$$

The retained bridge assigns no additional imagined task reward and has no success classifier, so $r_t^{\text{imag}} \equiv 0$ in this venue. The collector preserves the endpoint observation and potential and bootstraps from that endpoint. The implementation records this behavior with `bootstrap_on_truncation`. In both branches the shaped scalar is materialized before replay insertion and is not recomputed when the transition is sampled.

A.4 Residual Off-Policy Reinforcement Learning

The residual is trained off-policy in the RLPD (Ball et al. 2023) regime with TD3-style (Fujimoto, van Hoof, and Meger 2018) updates on a mixed replay buffer. The buffer stores the projected normalized action a_t defined in Section A.1 together with the base action a_t^b . This is the same action passed to the transition collector or dynamics model. The critic conditions on a_t , while the scaled residual mean μ_θ enters only the actor and demonstration anchor. An ensemble of N critics $\{Q_i\}_{i=1}^N$ (with $N = 10$) is trained toward an n -step clipped target that takes the minimum over two randomly chosen heads,

$$y_t = \sum_{j=0}^{n-1} \gamma^j \tilde{r}_{t+j} + \gamma^n (1 - d_{t:t+n}) \min_{i \in \mathcal{B}} \bar{Q}_i(o_{t+n}, z_{t+n}, a'_{t+n}),$$

where $d_{t:t+n}$ masks a terminal transition within the target window and $\mathcal{B}$ is a pair of random heads. The projected normalized target action is

$$a'_{t+n} = \text{clip}(a_{b,t+n} + \mu_{\bar{\theta}} + \epsilon, -1, 1),$$

where $\mu_{\bar{\theta}}$ is evaluated at $(o_{t+n}, a_{b,t+n}, z_{t+n})$. Target smoothing uses $\epsilon \sim \text{clip}(\mathcal{N}(0, \sigma), -c_{\text{clip}}, c_{\text{clip}})$ in scaled environment-action coordinates; ϵ is not rescaled by s_a ($\sigma = 0.05$, $c_{\text{clip}} = 0.3$; Table 5). The implementation’s critic loss averages the *absolute* TD error across the ensemble *before* squaring, $\mathbb{E}[(\frac{1}{N} \sum_i |Q_i(o, z, a) - y_t|)^2]$, rather than the per-head mean-squared error $\sum_i \mathbb{E}[(Q_i(o, z, a) - y_t)^2]$. The actor is updated less often than the critic and anchored to the demonstrations,

$$\mathcal{L}_\mu = -\mathbb{E}[\hat{Q}(o, z, a^\theta)] + \lambda_{\text{bc}} \mathbb{E}_{\mathcal{D}}[\|\mu_\theta - (a_{\text{demo}} - a_b)\|^2],$$

where $\hat{Q} = \frac{1}{N} \sum_{i=1}^N Q_i$ is the *online* ensemble mean over all N heads (the actor uses the mean, whereas the critic target above uses the min over two heads), and $a^\theta = \text{clip}(a_b + \mu_\theta(o, a_b, z), -1, 1)$ is the projected normalized action scored by the critic. The anchor is taken in scaled-action coordinates: the actor’s scaled residual mean μ_θ is regressed directly onto the base-relative demonstrator correction $a_{\text{demo}} - a_b$, *without* dividing by s_a and *without*

Algorithm 1 SHORE-RL residual training across validation venues

Input frozen π_b, ψ_b ; goal value V_{gc} and navigator π_h ; frozen bottleneck ϕ ; shaping F (stage bonus or $\gamma \Phi_V(s') - \Phi_V(s)$); collector $\mathcal{C}$; reward source R ; sampler ρ ; demonstrations $\mathcal{D}$
Output trained residual actor μ_θ and finetuned navigator π_h

```

1 initialize a zero-output residual actor and a 10-head critic ensemble
2 initialize replay stores with the offline transitions required by  $\rho$ 
3 for each collection round do
4    $\mathcal{T} \leftarrow \mathcal{C}(\pi_b, \mu_\theta, \pi_h, \psi_b)$ 
5   for each  $(o, z, a, a_b, r_{\text{env}}, o', d) \in \mathcal{T}$  do
6      $\tilde{r} \leftarrow R(r_{\text{env}}) + F(o, o')$ ; store the transition
7   update  $V_{\text{gc}}$  and  $\pi_h$  on equally mixed completed online and demonstration trajectories; keep  $\phi$  frozen
8   for each critic update prescribed by UTD do
9     sample a venue-specific mixed batch using  $\rho$ 
10    update all critics toward the  $n$ -step clipped target
11    on delayed steps, update the actor with  $Q$  and the demo anchor
12    soft-update actor and critic targets
13 return  $\mu_\theta, \pi_h$ 
```

clipping the target. This matches the normalized-coordinate form $\|\tanh(f_\theta) - (a_{\text{demo}} - a_b)/s_a\|^2$ up to the constant factor s_a^2 folded into λ_{bc} , except that the un-clipped target leaves a residual gradient toward $\tanh$ saturation whenever $|a_{\text{demo}} - a_b| > s_a$. The residual actor is zero-initialized as in Section A.1, and s_a keeps every explored action near the base. During collection the behavior policy *samples* the actor’s truncated Gaussian at exploration std $\sigma = 0.05$ in scaled coordinates rather than taking its mean—the exploration noise referenced in Section A.1—whereas the evaluation policy takes the deterministic mean.

A.5 Shared Update, Two Venue Instantiations

The actor and critic updates do not depend on where a transition originates, but the collector $\mathcal{C}$, reward source R , shaping F , and replay sampler ρ do. Algorithm 1 therefore states a venue-parameterized update rather than claiming that only one collection line changes. Table 2 lists the shared architecture together with the simulation defaults, and Table 1 lists what each venue sets differently.

B Simulation Venue

In simulation the environment is cheap to step, so the residual is trained *directly* against the simulator with no learned dynamics model. This section documents that venue end to end. The algorithm is the one specified in Section A, a frozen base policy with a small additive off-policy residual, a waypoint from a goal-conditioned value, and a dense potential; this section fixes its concrete configuration for the simulator, and the shared and venue-specific components are separated in Table 1.

B.1 Simulated Tasks and Success Criteria

Our DexMimicGen suite contains five dual-arm tasks (Jiang et al. 2025; Mandlekar et al. 2023). Four are compositionally

Component	Simulation	Real-robot imagination
Collector $\mathcal{C}$	simulator steps	bounded rollouts of frozen D
Env. reward	task reward	none ($r^{\text{imag}} \equiv 0$)
Shaping F	stage bonus or Φ_V	Φ_V only
Replay mix ρ	demo + online	recorded + imagined
Re-anchor	simulator reset	real-data start; depth ≤ 2
Other params	Table 5	Section C.5

Table 1: Method-level update and venue-specific instantiations. Collection, reward availability, and replay composition are venue-specific. The retained block command audited in Section C.5 additionally disables waypoint conditioning and online navigator/value updates; its recorded half of the 50/50 mixture is successful-demonstration replay.

Setting	Value
Residual output / scale s_a	$\mu_\theta = s_a \tanh(f_\theta)$, $s_a = 0.05$, zero-init
Critic ensemble	10 Q-nets; target min of 2 random heads
Critic / actor MLP	2 layers $\times$ 1024, ReLU, Layer-Norm
Discount γ / n -step	0.99 / 3
UTD / target Polyak	4 / 0.005
Target smoothing	0.05 (scaled coords), clip 0.3
Demo-BC coefficient λ_{bc}	0.1
Waypoint bottleneck dim	10 (radius $\sqrt{10}$)
Expectile τ / AWR β	0.7 / 1.0
actor_lr / critic_lr	1×10^{-6} / 1×10^{-4} (AdamW)

Table 2: Shared SHORE-RL architecture and simulation-default hyperparameters. Venue-specific scalar settings, collection, replay composition, and the retained block command’s disabled waypoint components are reported in Section C.5.

long-horizon and stage-structured: **PIECEASSEMBLY**, **POURING**, **LIFTTRAY**, and **THREADING**. The fifth, **CANSORT**, is a single-stage, same-benchmark control. We additionally evaluate the method on two single-arm tasks, one from **LIBERO-10** and one from **LIBERO-90** (Liu et al. 2023). Every DexMimicGen environment runs at a 20 Hz control rate. An episode is declared successful and terminated when its sparse task reward first reaches one; otherwise, it is truncated at the task-specific horizon in Table 3. For the four stage-structured tasks, a separate privileged detector maps each state to one of K discrete progress states, including the initial state 0, and supplies a bonus for newly attained stage transitions during training. **CANSORT** has no stage detector and uses no reward shaping. Representative execution sequences for all five tasks are shown in Figure 1.

LIBERO single-arm extension. We evaluate **LIBERO-10** task ID 8 (instruction: *put both moka pots on the stove*) and **LIBERO-90** task ID 57 (instruction: *pick up the cream cheese and put it in the tray*). Both tasks use a single arm, a 7-D

Task	Robots	act.	horizon	states K
PIECEASSEMBLY	2 $\times$ Panda	14	500	4
POURING	GR1	24	400	5
LIFTTRAY	2 $\times$ PandaDex	24	650	4
THREADING	2 $\times$ Panda	14	300	3
CANSORT, control	GR1	24	400	—

Table 3: Simulated DexMimicGen tasks. All tasks run at 20 Hz. *act.* is the action dimension; *horizon* is the maximum episode length used by our rollout wrapper; K counts the detector’s discrete progress states, including the initial state 0 except that **CANSORT** has no detector. Success occurs when the sparse reward first reaches one. Specifically, **PIECEASSEMBLY** requires piece 1 to be placed on the base and released, followed by piece 2 atop the assembly and released; **POURING** requires the ball to be transferred into the bowl and the bowl placed upright on the pad; **LIFTTRAY** requires both cubes to remain on the tray while the cubes and tray are lifted at least 0.10 m above the table, with the tray tilted by at most 30°; **THREADING** requires the needle center to lie within the ring radius; **CANSORT** requires the sampled red or blue can to be placed in the matching bin.

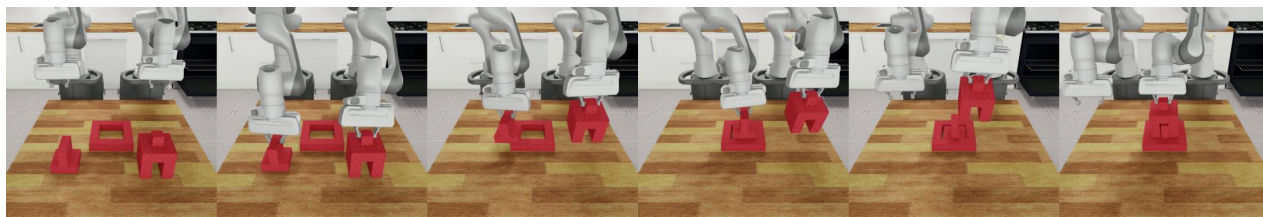
action space, and visual-proprioceptive observations comprising 8-D robot proprioception and two RGB views (an agent view and the wrist view `robot0_eye_in_hand`). No object state or other privileged simulator variable is provided to the policy. An episode terminates on the first task success or is truncated at 520 steps on **LIBERO-10** and 400 steps on **LIBERO-90**.

B.2 Base Policies and Frozen Features

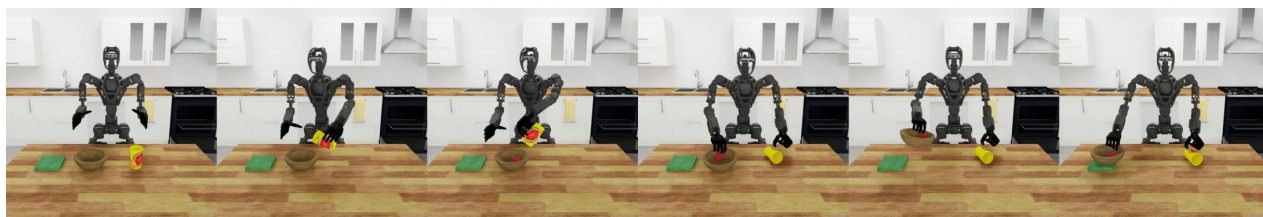
On the dual-arm tasks, the base policy is **ACT** (Zhao et al. 2023), fine-tuned *separately for each task* on its demonstrations and then frozen. On **LIBERO**, the base is a served π_0 VLA (Black et al. 2025b), which is likewise frozen. In the default and deployed configurations, the value modules read the base policy’s *frozen* encoder feature ψ_b together with proprioception, but no privileged object pose. The residual actor and critic do not share this feature extractor; they train their own multi-view encoder directly from the raw camera observations (Section A.4).

For **ACT**, ψ_b is the mean-pooled encoder-token feature (width 512). Concatenating proprioception gives an input dimension of 530 for **PIECEASSEMBLY** and **THREADING** (512+18), 548 for **POURING** (512+36), and 550 for **LIFTTRAY** (512+38). For π_0 , ψ_b is the last prefix token (width 2048), extracted server-side when the feature cache is built. Concatenating the standardized 8-D **LIBERO** proprioception gives a 2056-D input.

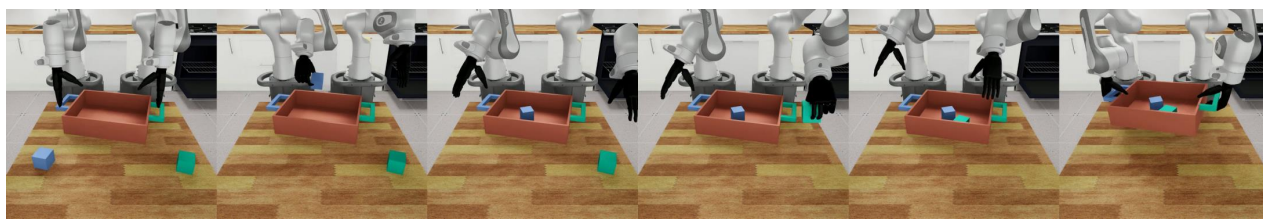
The frozen base emits action chunks that are executed open-loop for 10 steps between replans. We also evaluated a 30-step **LIBERO** setting as an execution-cadence diagnostic; it is not part of the headline comparison. Ablation D deliberately replaces the default feature with a privileged 30-D end-effector + object state (`eef_piece`) to provide an upper-bound reference. This ablation is the only use of priv-



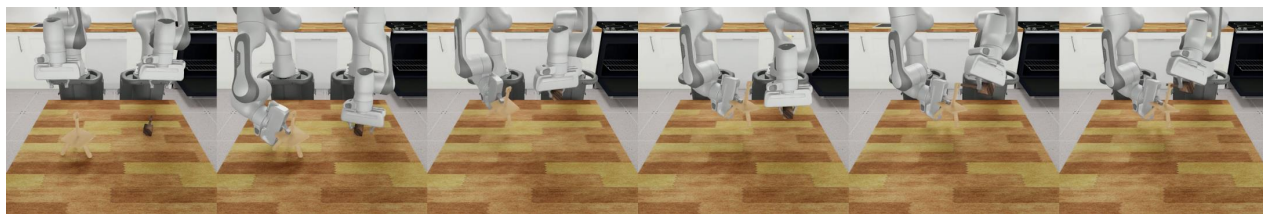
(a) PIECEASSEMBLY



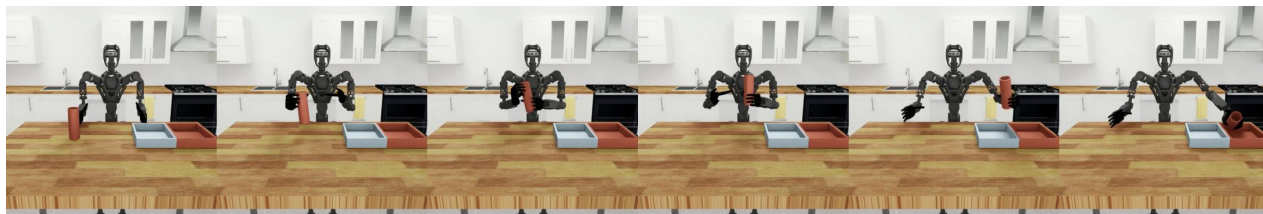
(b) POURING



(c) LIFTTRAY



(d) THREADING



(e) CANSORT

Figure 1: Representative execution sequences for the five simulated DexMimicGen tasks. Each row proceeds from left to right. Panels (a)–(d) show the four compositionally long-horizon tasks, while (e) shows the single-stage control task.

ileged object state; every deployed representation remains visual-proprioceptive.

B.3 Configuration and Hyperparameters

The goal-conditioned value V_{gc} and navigator π_h are pre-trained once per task from demonstrations using the frozen base-policy features ψ_b . In all waypoint-enabled runs, including the dual-arm and LIBERO runs, the two value heads and π_h are then jointly fine-tuned during residual RL while the bottleneck encoder ϕ remains frozen. These online updates use Adam with a learning rate of 10^{-5} for both modules, a batch size of 256 split evenly between demonstrations and online data, and begin after 2,000 online transitions; one joint update is applied per residual UTD block. The online navigator update aggregates the two value heads by their minimum.

The separate single-state value that supplies Φ_V is trained independently from demonstrations using only the current state, and remains frozen throughout residual training. Table 4 reports the value and navigator settings, including the architectural differences between the dual-arm and LIBERO instantiations. Table 5 gives the residual-RL configuration; its venue-independent core matches Table 2. Section C.5 separately identifies the optimizer settings reused in imagination and the collection settings that change. Together, these values instantiate the general update in Section A and the simulator version of Algorithm 1.

B.4 Baselines

We report a frozen-base reference, four external methods, and the component ablations below. A cross-method comparison is *matched* only if it shares the base, offline data, budget, execution horizon, evaluation harness, and seed policy of Section B.6. *BC base* is the frozen policy without RL; because the deterministic residual is initially zero (Section B.3), it is the leftmost evaluation point of each run under the same task-specific protocol. *Flat residual* (Ankile et al. 2026) removes the waypoint and shaping; we distinguish the collapse-control setting BC0 from the demo-anchored BC0.1 setting. *DSRL* (Wagenmaker et al. 2025) learns the correction in the same frozen base’s noise space. *IBRL* (Hu, Mirchandani, and Sadigh 2024) selects between base and learned actions by a Q -argmax, providing a soft anchor. We instantiate this selection rule on our RLPD/TD3 base and use the same evaluation harness as the other dual-arm comparisons. All IBRL runs reach 500k steps, with three seeds per task. *Full-policy IQL* (Kostrikov, Nair, and Levine 2022) instead fine-tunes the entire policy without a frozen base, isolating the residual parameterization.

B.5 Ablation Design

Table 6 gives the five-configuration structured component study. Within a task, all configurations share the codebase, frozen features, demonstrations, action scale, budget, and seed set. “Waypoint” includes online joint fine-tuning of the two value heads and navigator. “Stage” is the stage-derived training branch: it includes the direct bonus of Section A.3 and, where enabled, stage-balanced replay; it is not a potential-based reward.

Setting	Value
GC encoder/value (dual-arm)	MLPs 3×512 , LayerNorm+GELU
GC encoder/value (LIBERO)	MLPs 2×256 , LayerNorm+GELU
bottleneck input	$[g, s]$ dual-arm / g LIBERO
value heads / target	2 / per-head HIQL bootstrap
Single-state $V(s)$ MLP	2×256 , ReLU (separate net)
Waypoint bottleneck dim. z	10 (bottleneck radius $\sqrt{10}$)
Deployment navigator mean	raw dual-arm / radius- $\sqrt{10}$ renorm. on LIBERO only
Navigator π_h	2×256 , ReLU
AWR inverse temp. β	1.0 (weight clipped at 100)
Waypoint horizon k	15 dual-arm / 25 LIBERO
Expectile τ	0.7
Discount γ	0.99
Target EMA	0.005
Value goal mixture	current 0.2/future 0.5/random 0.3
future-state offset	geometric with parameter $1-\gamma$
Offline navigator target	clamp-to-goal; random-goal prob. 0.3
advantage aggregation	mean of V_1, V_2
Goal-conditioned reward	0 at the goal, -1 otherwise
Single-state $V(s)$ reward	1 at final step, 0 otherwise
Offline optimizer / lr	Adam / 3×10^{-4}
Offline steps / batch	50,000 / 256

Table 4: Value and navigator configuration. The dual-arm `hiqlv512` recipe uses 3×512 MLPs and `sg15` sets $k = 15$; the LIBERO instantiation uses 2×256 MLPs and $k = 25$. The single-state potential value is a separate state-only network.

Shaping form. Ablation B keeps waypoint+joint and BC0.1 fixed, and swaps only the direct stage bonus for the self-derived potential $\Phi_V(s)$. It therefore tests removal of the hand-designed detector without changing the rest of the method. The representation comparison between privileged `eef_piece` features and ACT or π_0 features is orthogonal.

Coverage. The five component configurations cover POURING and PIECEASSEMBLY, with three seeds per configuration and a common 4×10^5 -step budget. The shaping-form pair covers PIECEASSEMBLY and THREADING, with three seeds per configuration through 5×10^5 steps. CANSORT is excluded because its residual runs do not collapse and it has no stage decomposition.

B.6 Evaluation Protocol

All SHORE-RL and in-codebase ablation configurations use a common evaluator within each task. We call a cross-method comparison *matched* only when the base checkpoint, offline demonstrations, execution horizon, success criterion, training budget, and seed policy also agree. For the dual-arm tasks, evaluation uses an execution horizon of 10, eight parallel environments, and 50 deterministic episodes per checkpoint; LIBERO uses eight environments and 10 episodes. For a success probability p , the binomial standard error of a 50-episode success-rate estimate is $\sqrt{p(1-p)/50} \approx 0.057$

Setting	Value
Regime	RLPD, TD3-style updates
Critic ensemble	10 Q-nets; target = min of 2 random heads
Critic / actor MLP	2 layers $\times$ 1024, ReLU, LayerNorm
Residual actor output	$\mu_\theta = s_a \tanh(f_\theta)$; final layer zero-init
Action scale s_a	0.05
Discount γ / n -step	0.99 / 3
Update-to-data (UTD)	4 (actor updated every 4th critic step)
Target Polyak τ	0.005
Exploration / target noise	0.05 (scaled coords), smoothing clip 0.3
actor_lr / critic_lr	1×10^{-6} / 1×10^{-4} (AdamW)
Batch / replay buffer	256 / 2×10^5
Offline demo fraction	0.5 (RLPD demo mixing)
Demo-BC coefficient	0.1 (MSE in scaled-action coords; no $1/s_a$, no clip)
Learning starts	10^4
Total env. steps	5×10^5
Potential $\Phi_V = cV(s)$	$c = \frac{\max(K-1, 1)}{\max(V_{\max} - V_{\min}, 10^{-6})}$ for K stages; multiplier 1.0
Stage bonus (main)	direct +1 per stage advance, stage-balanced replay
Gradient clip	1.0
Eval	50 ep. (DexMimicGen) / 10 (LIBERO) 8 envs, deterministic ($\sigma=0$)

Table 5: Residual off-policy RL configuration in simulation. The update combines RLPD’s 10-head critic ensemble and 50/50 demonstration mixing with TD3-style target smoothing and delayed actor updates. The residual actor’s final layer is zero-initialized, so its deterministic output is exactly zero at step 0; the leftmost evaluation therefore measures the frozen base (Section B.4).

at $p = 0.8$; this within-checkpoint uncertainty is distinct from the across-seed s.e.m. bands in the learning curves. The nominal online budget is 5×10^5 environment steps for the main DexMimicGen comparison and 4×10^5 environment steps for LIBERO; component Ablation A uses the matched 4×10^5 -step budget stated above. Since the residual mean is zero before training, the step-0 evaluation also provides the frozen-base reference.

Method-specific anchoring parameters—our action scale, DSRL’s noise scale, and IBRL’s exploration scale—are calibrated only within the protocol used for the corresponding comparison. The LIBERO DSRL overlay is the sole exception: it uses DSRL’s query-20 harness and is therefore indicative rather than strictly matched. The IBRL comparison uses the matched dual-arm harness and the full 500k-step budget.

Reported metric. For each seed, the *steady-state* success rate is the mean over its final 20% of evaluation checkpoints. This window always contains at least one checkpoint. We then report the mean and standard error of these seed-level values. Learning curves instead aggregate seeds at each recorded checkpoint and show a ± 1 s.e.m. band when at least

Configuration	waypoint	BC	stage	Removed
Full (SHORE-RL)	on	0.1	on	—
– stage	on	0.1	off	stage branch
– waypoint	off	0.1	on	waypoint
– waypoint – stage	off	0.1	off	waypoint + stage
waypoint only	on	0	off	BC + stage branch

Table 6: Structured component ablation (Ablation A). **Full** uses waypoint+joint fine-tuning, demo-BC, and the stage-derived training branch; each subsequent configuration removes the component(s) named in the last column.

two seeds are available. Diagnostic runs outside the matched main comparisons are shown only over their observed ranges; their terminal windows are neither extrapolated nor treated as budget-matched results.

B.7 Results, Run Inventory, and Provenance

The main paper reports the compact steady-state table and learning curves. SHORE-RL and all four main baselines—the flat residual (ResFit), DSRL, IBRL, and IQL—use three seeds per method on all four long-horizon tasks and reach 500k steps. The CANSORT diagnostic uses 2–3 seeds per method, including three completed IBRL seeds. Component ablation (A) covers POURING and PIECEASSEMBLY with three 400k-step seeds per configuration. Shaping comparison (B) uses three 500k-step seeds per configuration on PIECEASSEMBLY and THREADING. The LIBERO-10 Task 8 and LIBERO-90 Task 57 studies use three 400k-step seeds per method; the DSRL comparison uses the separate harness described below.

Inclusion and provenance. The included dual-arm SHORE-RL, flat-residual, and ablation runs use the same execution horizon (10) and eight-environment evaluator; legacy ablations evaluated with one or four environments are excluded. Frozen-base references come only from step-0 evaluations in the same harness and are never pooled across evaluator settings or codebases.

Removing the waypoint also removes its online joint update and is therefore treated as one bundled mechanism. A second, task-dependent caveat applies to the structured component study: on POURING and PIECEASSEMBLY, the –stage configurations also disable stage-balanced replay and thus remove all stage-derived training information. The shaping-form comparison is cleaner: all three seeds reach 500k, and only the direct stage bonus is replaced by the self-derived potential.

Coverage caveats. The main long-horizon comparison is complete and matched at three seeds and 500k steps for every method. IBRL’s final-window performance is 0.00 ± 0.00 on POURING, LIFTTRAY, and THREADING, and 0.10 ± 0.05 on PIECEASSEMBLY. The smaller CANSORT diagnostic uses the available 2–3 seeds per method, and the LIBERO DSRL overlay remains a separate-harness comparison, as noted in Section B.6.

B.8 Limitations of the Simulation Venue

Statistical coverage. The component ablation includes three matched 400k-step seeds per configuration on POURING and PIECEASSEMBLY, and the shaping-form comparison includes three full-budget seeds per configuration on PIECEASSEMBLY and THREADING. Their task coverage nevertheless remains narrow: the evidence supports the reported component interactions and the self-derived potential on these tested tasks, not universal equivalence or task-independent effects. The smaller CANSORT study is therefore treated as a diagnostic rather than uniformly powered evidence.

Privileged task structure. The main simulation result uses a per-stage bonus computed from a task-specific simulator predicate and thus applies only when a reliable stage decomposition is available. The self-derived $V(s)$ potential removes this annotation requirement, but its quality depends on the frozen value estimate and it need not recover a useful progress signal on every task. This is why the real-robot venue uses the $V(s)$ form throughout, while the simulation study reports both formulations.

Cross-method comparability and benchmark scope. The main dual-arm comparisons are matched within our evaluator. On LIBERO, however, the DSRL overlay uses its own evaluation harness, so absolute-success differences are indicative rather than strictly controlled. The extension covers only LIBERO-10 Task 8 and LIBERO-90 Task 57 rather than the full LIBERO benchmark; claims about broader benchmark transfer would require matched evaluation on additional tasks.

C Real-Robot Venue

Everything in Section B trains the residual against a cheap-to-step simulator. For hardware evaluation we retain the residual parameterization and actor-critic objective of Section A, but instantiate its collector and replay sampler with a learned dynamics model finetuned on our own robot data. This section documents that venue: the platform and its interfaces (C.1), the three tasks and their success criteria (C.2), the demonstration and rollout data (C.3), the dynamics model (C.4), the imagination loop the residual is trained in (C.5), the protocol used to evaluate the resulting residual on the physical robot (C.6), and the limitations specific to this venue (C.8).

C.1 Real-Robot Platform and Control

Robot. The real-robot venue uses a full-body robot; control is over its two 7-DoF arms, each terminated by a parallel gripper. Commands are joint-space targets, so the action space is 16-dimensional, $(7 \text{ joints} + 1 \text{ gripper}) \times 2$. The robot controller consumes one joint-space command per step at 30 Hz (matching the camera rate), and commands are absolute joint-position targets rather than deltas.

Perception. Perception is image-only: three RGB cameras stream 848×480 video at 30 Hz, consisting of one head-mounted view (`top_head`) and two wrist-mounted views (`hand_left`, `hand_right`). The same three views are consumed by every component that needs to see the scene: the base policies, the frozen encoder ψ_b that supplies the

value’s state representation, and the dynamics model. No component receives a signal the deployed robot does not have, because there is no motion capture, no object pose, and no privileged state anywhere in this venue.

Base policy and action chunking. Each task’s base policy is a served $\pi_{0.5}$ -style VLA (Yu et al. 2026; Black et al. 2025a) that emits 50-step action chunks, executed open-loop within each chunk. Internally the VLA’s action vector is zero-padded to its native width of 32 dimensions, of which the leading 16 are the platform’s commands; the padding is stripped before execution and before any residual is added.

What runs at deployment. At deployment the robot runs the task’s frozen base policy, the navigator on the frozen base features, and the residual actor on its separate trained multi-view image encoder together with proprioception, the base action, and the waypoint. The dynamics model is a training-time device only and is never executed at inference time, so the deployed system carries no video dynamics model; its additional computation is the residual encoder and actor plus the navigator.

C.2 Tasks and Success Criteria

We evaluate on three tabletop bimanual tasks. The robot controller and cameras operate at 30 Hz during physical evaluation. The dynamics-model training data uses 30 Hz for BUILD_BLOCK and PICK_CUP, while the PICK_PAPER_ROLL domain is made internally uniform at 20 Hz by resampling its frozen-base rollouts (Section C.4.2). Episodes end when the task is completed or its task-specific timeout is reached. **[TODO: report the physical-evaluation timeout and resulting trial-duration range for each task.]** For each task we quote the language instruction verbatim as it appears in the dataset, because that same string is the caption conditioning the dynamics model (Section C.4), and because a mismatch between the two would silently change what the model is conditioned on. Representative physical-robot executions are shown in Figure 2.

PICK_PAPER_ROLL The instruction is `put the paper roll on the holder`. The robot grasps a paper roll from the tabletop and places it onto an upright holder. A key challenge is that the roll is a rolling object: a grasp that is not centered displaces it rather than securing it, and the resulting error compounds into the placement. The representative successful sequence in Figure 2 starts with the roll upright and uses one gripper to transport it directly to the holder; no inter-arm handover occurs. *Success*: the holder’s vertical peg passes through the roll’s central opening, the roll rests on the holder base, and the gripper has released and withdrawn.

PICK_CUP The instruction is `pick cup`. This is the Cup Stacking task in the main paper. Ten upright paper cups start distributed on the tabletop; the robot rearranges them into a four-level 4–3–2–1 stack, so reorienting a fallen or sideways cup is not part of the demonstrated task. *Success*: all ten cups form the required four-level structure and remain stable after the grippers release and withdraw.

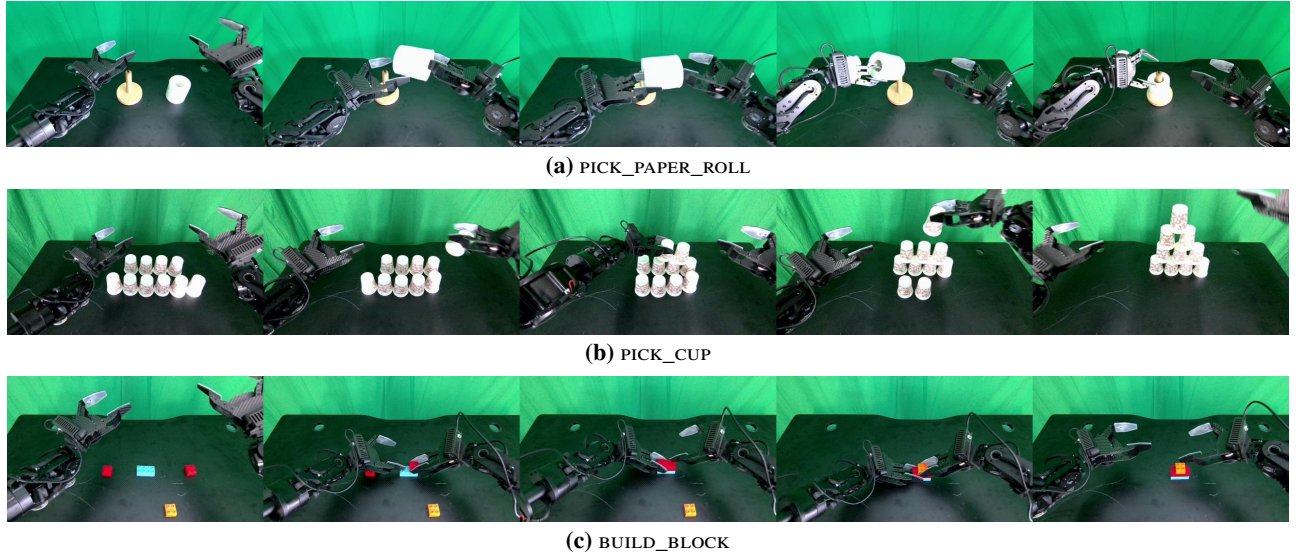


Figure 2: Representative execution sequences for the three real-robot tasks, shown from left to right: (a) placing a paper roll on its holder, (b) manipulating and arranging the cups, and (c) assembling the colored blocks.

BUILD_BLOCK The instruction is `build block`. This is an assembly-style task. The representative sequence starts from four loose pieces (two red, one cyan, and one yellow) and builds a three-piece vertical target: cyan base, red middle, and yellow top, leaving the second red piece unused as a distractor. *Success*: the three target pieces form this assembly and it remains intact after the grippers release and withdraw. **[TODO: confirm whether this color order is fixed across the entire evaluation set and whether each arm is assigned particular pieces.]**

Randomization. Object placements vary across episodes. Physical evaluation uses a fixed initial-state set shared across the compared conditions. **[TODO: confirm the training randomization protocol—which objects are randomized, over what region, and whether the robot’s initial pose is randomized—and report how the fixed evaluation set was constructed and whether it is held out from the training-placement distribution.]**

Why binary success. We score each trial with a single binary criterion rather than a per-stage rubric. This keeps the real-robot metric identical in kind to the simulated one and avoids introducing stage labels into a venue whose entire point is that it runs without them: the shaping potential here is the self-derived single-state value, precisely because no stage detector is available on hardware.

C.3 Demonstration and Rollout Data

Two kinds of data are collected on the physical robot, and they enter the pipeline at different points.

Expert demonstrations. For each task we collect about 1,000 successful human-teleoperation episodes, stored in LeRobot format with per-frame images, joint states, and actions. These train the base policies: we finetune the $\pi_{0.5}$

Task	successful teleoperation	base rollout	Hz
BUILD_BLOCK	$\sim 1,000$	50	30
PICK_CUP	$\sim 1,000$	51	30
PICK_PAPER_ROLL	$\sim 1,000$	53	20

Table 7: Real-robot data scale, per task. Each task is one dynamics-model domain (`block`, `cup`, and `paper`, respectively; Section C.4.2). Successful human teleoperation trains the task-specific base; recorded frozen-base rollouts enter only dynamics-model finetuning. The main paper’s approximately 1,000 successful demonstrations per task is the authoritative collection count. **[TODO: reconcile the filtered local pipeline inventory with the full collection and then report exact per-task demonstration counts and merged frame totals.]**

VLA *separately per task*, giving one base policy per task, each frozen from that point on.

Base-policy rollouts. We then deploy each frozen base on the physical robot and record its rollouts. These are mixed-outcome by construction, in that they contain exactly the residual failures that the base policy has not overcome, and they are used only for the dynamics model, never for base-policy training. This is deliberate: the model must be able to render the failure modes the residual is being trained to avoid, and a model finetuned on success-only data cannot, because it has never seen one.

C.4 Dynamics Model

C.4.1 Architecture and Action Conditioning The dynamics model D predicts the next H multi-view frames from an observation history and an action chunk. We instantiate it as an action-conditioned multi-view video-diffusion model

and finetune it from a publicly released checkpoint pretrained on large-scale robot-manipulation video (Yang et al. 2026), following that work’s training recipe.

The backbone is an LTX-Video (HaCohen et al. 2025) stack, comprising a T5 (Raffel et al. 2020) text encoder, a causal video VAE, and a 28-layer diffusion transformer (32 attention heads of dimension 64, cross-attention width 2,048, 128 latent channels), and it is trained with a flow-matching objective (Lipman et al. 2023) under logit-normal timestep weighting. Each training sample is 3 views $\times$ [4 history + 25 predicted] frames at 256×192 , downsampled from the native 848×480 , paired with the 50-step action chunk that spans the predicted window.

During dynamics-model finetuning, actions are normalized using the statistics specified by the training configuration. The retained `BUILD_BLOCK` imagination launcher, however, does not provide an `action_norm.json`. Its bridge therefore uses fixed per-dimension bounds of $[-1, 1]$, samples action indices $0, 2, \dots, 48$ from each 50×16 chunk, zero-pads the 16 channels to 30, and sends a bf16 tensor of shape $1 \times 25 \times 30$. The stored step-18,000 checkpoint contains the action-conditioning path, but its exact initialization provenance is not retained locally. The action embedding is added to the caption embeddings that drive cross-attention, a comparatively weak form of conditioning; action controllability must therefore be tested rather than assumed (Section C.4.4).

C.4.2 Mixed-Quality Training Data and Shortcut Controls The available finetuning configuration specifies a *single* dynamics model and a joint full-parameter update over the pooled data of all three tasks. The asymmetry with the base policies, which are one per task against a single shared dynamics model, is deliberate: predicting how a scene responds to an action chunk is largely task-independent and benefits from pooling, whereas the policies do not.

The data is organized as three domains, one per task, each containing that task’s expert demonstrations *and* that task’s base-policy rollouts merged into a single dataset (Table 7). Because the model has no notion of success or failure, its objective being pure video prediction, mixing the two is what teaches it to render failure at all.

That mixing, however, creates an opportunity for the model to learn a shortcut instead of dynamics, and we control for two such shortcuts explicitly.

(i) Frame rate, within a domain. The `PICK_PAPER_ROLL` expert data is recorded at 20 Hz while its failure rollouts were recorded at 30 Hz. The loader samples windows by *frame count*, not by wall-clock duration, and the model is never told the frame rate, since its temporal position encoding is the bare frame index. A within-domain frame-rate difference would therefore make inter-frame motion magnitude collinear with success versus failure, letting the model infer the outcome from apparent speed rather than from the action-conditioned dynamics, which is exactly the capability the negative probe of Section C.4.4 is meant to test. We resample the failure rollouts to 20 Hz so that the domain is internally uniform. The remaining *across*-domain difference (20 Hz for `paper` versus 30 Hz for `block` and `cup`) is left as is: it is collinear

Setting	Value
Initialization	released pretrained checkpoint
Objective	flow matching (logit-normal weighting)
Views	3 (head, left wrist, right wrist)
Frames per sample	4 history + 25 predicted
Resolution	256×192
Action chunk	50 steps, per-domain normalized
Optimizer	AdamW, $\beta = (0.9, 0.95)$
Learning rate	1×10^{-4} , constant
Warmup steps	2,000
LR scaling by batch	none
Precision	bf16, gradient checkpointing
Sharding	DeepSpeed ZeRO-2
Batch size	4 per GPU (48 GB) / 8 per GPU (96 GB)
Training steps	18,000 (checkpoint used downstream)
Caption dropout	0.06
First-frame noise	0.2
Validation	every 1,000 steps (per-domain videos)
Checkpoint	every 2,000 steps

Table 8: Available finetuning configuration associated with the step-18,000 artifact loaded by the retained `BUILD_BLOCK` service. The imagination launcher performs inference only and does not itself record or verify the served checkpoint identity or pooled-data provenance. **[TODO: report the dynamics-model finetuning hardware and wall-clock time.]**

with the task, not with the outcome, and task identity is already supplied to the model through the caption.

(ii) Caption, within a domain. For the same reason, all episodes within a domain carry one identical caption, so text cannot serve as a success/failure indicator. This required unifying the `block` domain, whose expert and rollout splits had been captioned differently, onto the single string `build block`.

Scope of the retained launcher. The retained block imagination launcher does not finetune the dynamics model. It connects the learner to a model that must already be served at port 9000. The retained service log shows a load from `block_infer.yaml` and the step-18,000 diffusion checkpoint. However, the websocket metadata contains only a generic model name and the camera list, and the run’s `bridge_run_config.json` does not record the server configuration or checkpoint digest. The launch record therefore establishes the inference path below, but not the pooled three-domain training provenance or an immutable run-level binding to that checkpoint. Those require a separate dynamics-training record.

C.4.3 Finetuning Configuration The available finetuning configuration specifies a single joint, full-parameter update over the pooled data of all three domains. Table 8 lists its settings.

C.4.4 Qualitative Inspection and Available Monitoring The dynamics model is the environment in this venue, so failures of the dynamics model become failure modes of the residual. A complete validation protocol should therefore

Retained block inference field	Value
Service configuration	block_infer.yaml
Loaded model	diffusion checkpoint, step 18,000
Checkpoint SHA-256	9b07acec...69f8a
Transformer parameters	1,990,672,572
Prompt	build block
History tensor	$3 \times 3 \times 4 \times 192 \times 256$
Action-token tensor	$1 \times 25 \times 30$ (bf16)
Inference steps	10
Returned prediction	$3 \times 3 \times 25 \times 192 \times 256$

Table 9: Code-backed inference contract for the retained block launch with training GPU 7 and seed 0. The service returns 29 frames per view including the four conditioning frames; the bridge removes those frames and retains the 25 predicted frames reported here. The digest is the locally verified SHA-256 prefix and suffix of the loaded model artifact.

cover three complementary axes before any policy training. Figure 3 reports the currently available qualitative evidence for the first axis on one successful recorded episode from each task. The retained launcher does *not* implement a held-out dynamics-model validation: its scheduled evaluator instead rolls the current residual policy inside the same imagination environment and start-state pool used for training. The held-out aggregate evaluation and the remaining two axes are therefore left explicit below.

(i) Imagined-rollout quality. Given a real observation history and the ground-truth action chunk that follows it, the imagined H -step future is compared against the recorded future. For each task, Figure 3 shows representative, temporally corresponding head-camera observations from the recorded and predicted futures. The displayed moments are selected separately for each task, rather than using a shared fixed-step schedule. Agreement in object layout, manipulator motion, and task progression provides a direct qualitative check of rollout quality. The dynamics model predicts all three camera views, but only the head view is shown here for readability.

(ii) Action controllability, including a negative probe. The same observation history under different action chunks must yield *diverging* futures; a model that ignores its action input is a video prior, not a dynamics model, and would make the residual gradient meaningless. The sharper form of this test is the negative probe: we feed deliberately perturbed actions, such as translating the grasp segment by a few centimeters or closing the gripper early. The imagined video must show the corresponding failure rather than a hallucinated success; examples include a missed grasp or a slip. Passing this probe is the entire reason the mixed-quality data of Section C.4.2 exists; failing it means the failure fraction or the training budget is insufficient.

(iii) Potential-trajectory consistency. Finally we check the quantity the residual actually optimizes, rather than pixels: $V(\psi(\hat{o}_{t+k}))$ must track $V(\psi(o_{t+k}))$ over k . A model can be visually plausible and still be useless here if the re-encoded imagined frames land where the potential misreads

Reported step	Mean final proxy	Population SD
50	0.617	0.194
50k	0.560	0.218
100k	0.317	0.571
150k	0.473	0.348
200k	0.396	0.413
250k	0.262	0.370
300k	0.471	0.288
350k	0.329	0.474
400k	0.463	0.473
450k	0.442	0.303
500k	0.496	0.354

Table 10: Training-time imagined value-proxy record for the retained BUILD_BLOCK, seed-0 run (10 rollouts per checkpoint). The evaluator samples from the training start-state pool, records no episode/frame identifier or real future, and applies no calibrated success threshold. Accordingly, these values are monitoring diagnostics only.

them, so this is the axis that most directly predicts whether imagination training transfers.

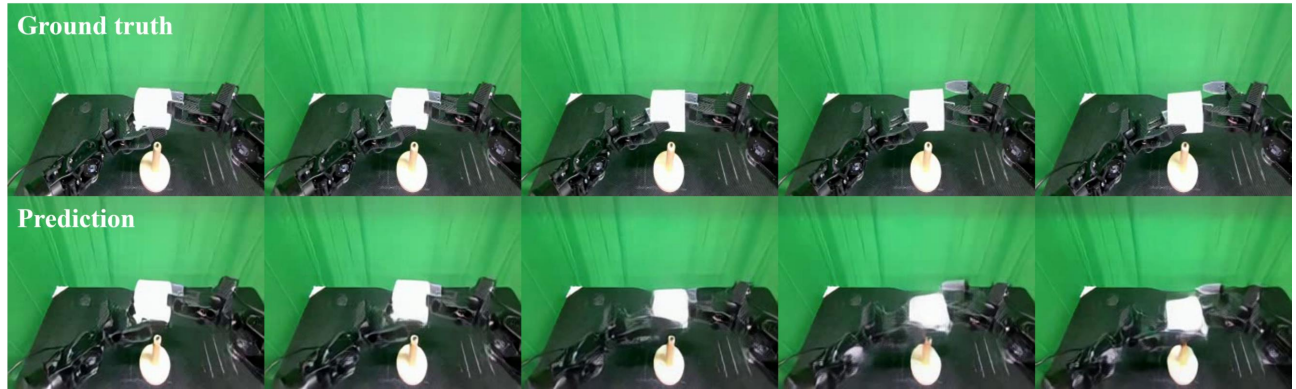
[TODO: P0: report the held-out split size and a quantitative rollout metric for (i), then run and report the action-perturbation probe in (ii) and the potential-trajectory comparison in (iii).]

What the retained launch actually records. At the first collected chunk (step 50) and then every 50,000 reported control steps through step 500,000, the evaluator saves a checkpoint and generates ten two-segment imagined rollouts. Each rollout contains 50 predicted frames. The resulting JSONL artifact contains 110 trajectories and 5,500 frame scores from a *separate* frozen value model. Table 10 reports the final-frame proxy. It is a continuous training-time diagnostic, not a success classifier, a real-robot success rate, or a held-out world-model metric. The evaluator deliberately returns `eval/success_rate=0.0` only as a checkpointing placeholder; that field must not be read as an observed 0% success rate.

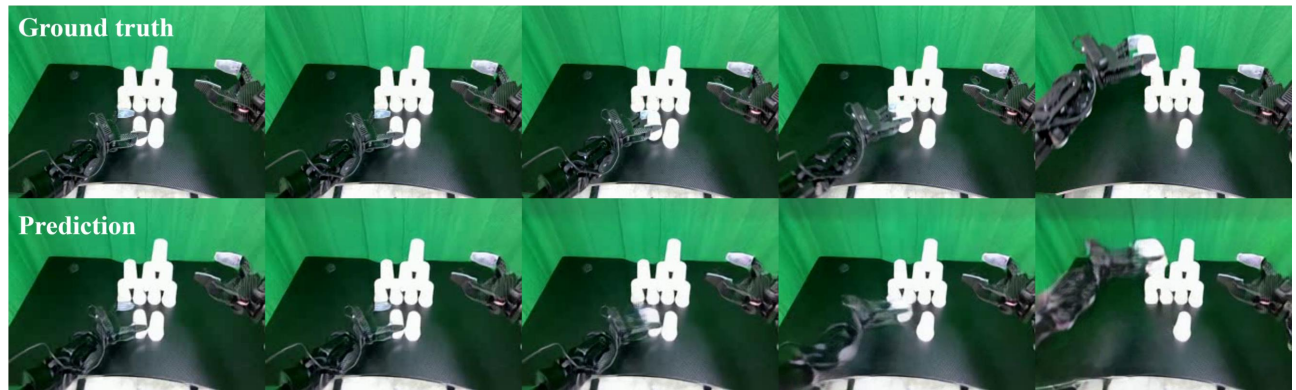
C.5 Residual RL in Imagination

Exact scope of the retained block command. The following implementation record is for the block launcher with training GPU 7 and seed 0. Despite the output-directory label containing “shore,” its recorded trainer configuration uses the raw actor and disables subgoal conditioning, stage conditioning, relabeling, online value finetuning, and online high-actor finetuning. Thus this exact artifact is a flat, non-waypoint-conditioned residual run: it contains no z_t , no online V_{gc} update, and no navigator π_h update. A waypoint-conditioned SHORE-RL real-robot result cannot be attributed to this command without a separate matching launch configuration and artifact.

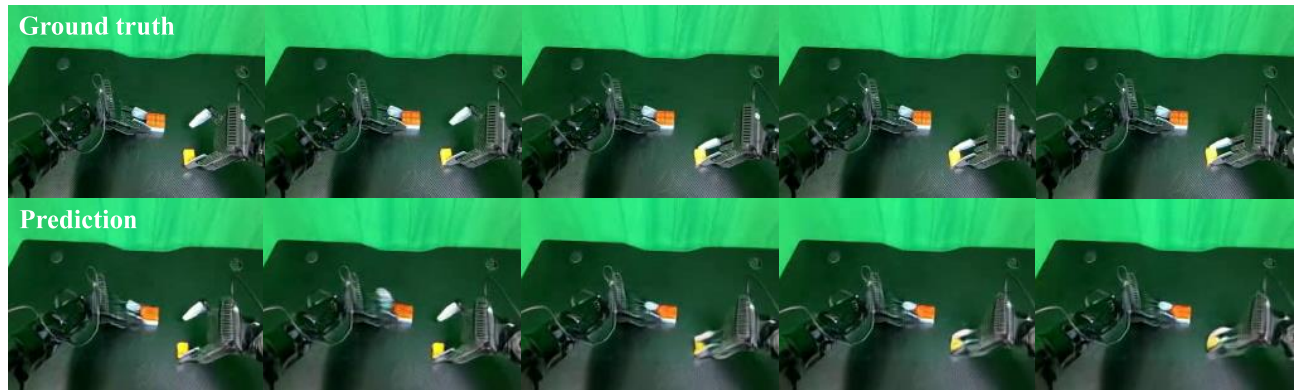
Synchronous segment generation and replay. A video-diffusion model is too slow to run at the robot’s control frequency, so the implementation exposes it as a chunk-level rather than control-step environment. The frozen base and



(a) PICK_PAPER_ROLL



(b) PICK_CUP



(c) BUILD_BLOCK

Figure 3: **Qualitative action-conditioned rollout comparison.** One successful recorded episode is shown for each real-robot task: (a) PICK_PAPER_ROLL, (b) PICK_CUP, and (c) BUILD_BLOCK. Each panel compares representative head-camera observations from the recorded future (top) and the dynamics-model prediction (bottom), conditioned on the same observation history and recorded action sequence. The displayed moments are selected separately for each task and do not follow a shared fixed-step schedule. These examples provide qualitative evidence for rollout correspondence only. They are not generated by the residual-training launcher and do not replace the pending held-out quantitative metric, action-perturbation probe, or potential-trajectory comparison.

flat residual produce a 50×16 action chunk. After the outer wrapper accumulates all 50 commands, D consumes the sub-sampled action tokens together with four history frames and predicts 25 future frames for each of the three camera views. At the block dataset rate of 30 Hz, one action chunk spans 1.667 s and the predicted frames are at 15 Hz. The final predicted frame is re-encoded by the frozen base, scored once, and stored immediately as one transition in the online imagination replay. Generation and TD3 (Fujimoto, van Hoof, and Meger 2018) updating therefore alternate synchronously; there is no periodic K -update refresh or separately generated batch of M rollouts.

Each seed drawn from the recorded robot data is advanced for at most two generated segments. At the second-segment boundary, the collector preserves the generated endpoint as the replay bootstrap observation, marks the boundary as an artificial truncation, and resets from a new recorded seed. Seeds are sampled uniformly over the pooled episode list containing 295 successful demonstrations and 50 frozen-base failure rollouts, followed by a uniform valid four-frame window within the selected episode. The recorded failure transitions themselves are not inserted into the training batch. This bounded recursion gives each imagined rollout 100 actions and 50 predicted frames (3.333 s at 30 Hz) before it is re-anchored to real data (Janner et al. 2019). The sampler uses an unseeded `numpy.random.default_rng()` and does not record the sampled episode/frame identifiers, so the command’s seed 0 does not make the exact start-state sequence replayable.

The online replay has capacity 200,000 chunk transitions. A separate offline replay is built from the retained 295 successful `BUILD_BLOCK` demonstrations. It contains 6,989 non-overlapping 50-step transitions plus a final aligned chunk where needed; no episode was skipped as too short. **[TODO: reconcile this filtered replay artifact with the full demonstration collection summarized in Table 7.]** Every update samples 128 imagined and 128 demonstration transitions, giving batch size 256 and an offline fraction of 0.5. Training starts after 10,000 reported control steps, equal to 200 generated chunk transitions, and first performs 10,000 critic-only warmup updates. Thereafter the update-to-data ratio is 4, with a one-step target, $\gamma = 0.995$, residual scale $s_a = 0.2$, and demonstration-BC coefficient 0.1. The reported 500,000-control-step checkpoint corresponds to 10,000 generated chunk transitions. The launcher assigns four single-GPU processes: the dynamics-model service on GPU 0, frozen-base service on GPU 1, monitoring-only advantage proxy on GPU 6, and seed-0 learner on GPU 7. The retained online W&B run `eegyfmzmz` (code commit `b22fcbda...b9a4`) records 18,798 s (5.22 h) from W&B initialization through step 500,000; offline cache construction occurred before that timer. Eleven persistent checkpoints were saved at steps 50, 50k, 100k, ..., 500k, in addition to a rolling latest checkpoint. The final retained checkpoint is step 500,000. **[TODO: report matching provenance and runtime for the other task runs.]** This loop implements synchronous chunk-level residual TD3 with a two-segment imagination collector and a 50/50 demonstration–imagination replay mixture.

No task reward; self-derived endpoint potential only. The hand-designed per-stage bonus used in the main simulated results (Section B.3) reads simulator predicates that do not exist here. In the bridge, the additive task-reward term is identically zero, $r_t^{\text{imag}} \equiv 0$, and each completed imagined chunk receives

$$\tilde{r}_t^{\text{imag}} = 0 + \gamma \Phi_V(\hat{e}_{t+1}) - \Phi_V(e_t),$$

where $\gamma = 0.995$ and Φ_V is a frozen ValueMLP evaluated on the concatenation of mean-pooled frozen-base prefix features and proprioception. The potential checkpoint has SHA-256 `1eb54786...40411b`. There is no imagined success classifier and no task-success terminal: `terminated` is always false, while the second-segment boundary sets `truncated=true`. The successful-demonstration replay uses the same potential-only endpoint reward; only its final recorded chunk has `done=true` and masks the critic bootstrap. An artificial imagination boundary instead stores `done=false` and bootstraps from its preserved generated endpoint. The trainer’s `reward_shaping=none` prevents a second shaping pass; the logged `potential_source=stage` option is therefore inactive and does not indicate use of a stage detector. No physical environment transition is executed during residual training.

What is frozen and what is trained. For this command, the task-specific base policy and its feature encoder ψ_b , the dynamics model D , the single-state value supplying Φ_V , and the separate advantage-proxy monitor are frozen. Only the flat residual actor and critic are trained. The waypoint bottleneck, goal-conditioned value, and navigator are not instantiated in this run.

C.6 Real-Robot Evaluation Protocol

The residual trained in imagination is evaluated on the physical robot. We compare three conditions: the task’s frozen base, base plus the flat ResFit residual, and base plus the waypoint-conditioned SHORE-RL residual. Within each task all conditions use the same frozen base, observation interface, controller, and fixed physical initial-state set; the two adapted conditions differ in the residual method.

Trials. 50 trials per cell, using the fixed physical initial-state set stated in the main paper for all compared conditions. The available aggregate record does not preserve the per-trial initial-state assignment, trial order, or scorer metadata.

Paired initial states. The shared fixed set establishes set-level control of initial conditions, but it does not by itself establish a one-to-one pairing of individual trials. **[TODO: report the set’s exact composition and reset procedure, and recover the trial-level initial-state labels needed to determine whether paired inference is valid.]**

Interleaving. **[TODO: report whether conditions alternated within each session or were run in blocks, together with the session order needed to assess lighting, gripper-wear, and calibration drift.]**

Blind scoring. [TODO: report who scored the recorded trials, whether scoring was blind to the condition, and how disagreements or ambiguous trials were resolved.]

Reporting. For each cell we report successes out of 50, the empirical success rate, and its Wilson score 95% interval, which is better behaved than the normal approximation near the extremes where several cells sit. The available result record preserves these marginal counts but not the trial-level pairing labels, so Table 11 reports only a descriptive absolute difference. [TODO: recover the paired trial-level outcomes and report the two discordant-pair counts, a paired confidence interval or exact McNemar test, and the treatment of aborted or invalid trials.] We retain a separate failure-analysis table below; its categories and counts remain unresolved because they cannot be recovered from marginal success rates.

C.7 Real-Robot Results and Failure Analysis

The main paper reports learning curves at six checkpoints from step 0 through 500k. The retained aggregate physical-robot record contains 50 trials per configuration and resolves the step-0 base and step-500k ResFit and SHORE-RL marginal success rates below; the dynamics-model validation metrics of Section C.4.4 and paired inferential analysis remain to be reported.

At step 500k, SHORE-RL improves over the frozen base by 10 additional successes on PICK_PAPER_ROLL, 9 on BUILD_BLOCK, and 10 on PICK_CUP. ResFit gains 3 successes on PICK_PAPER_ROLL but loses 5 relative to the base on each of BUILD_BLOCK and PICK_CUP. [TODO: report the paired test, session count and dates, invalid-trial count, and whether each task’s fixed initial-state set was pre-registered.]

C.8 Limitations of the Real-Robot Venue

The environment is a model. The residual is optimized against D , not against the robot, so any systematic error in D becomes a systematic bias in the learned residual. We bound the compounding with a recursion depth of two, but the retained command does not execute the held-out tests specified in Section C.4.4. The honest reading of this venue is therefore that it trades an unbiased but unaffordable environment for a biased but cheap one whose quantitative validation remains incomplete.

The learned potential is not physical feedback. The imagined objective contains only changes in the learned potential; there is no independently observed task reward or success terminal to correct it. A mis-calibrated $V(s)$ can therefore reinforce model bias, which makes the validation axes in Section C.4.4 load-bearing rather than a nicety.

Weak action conditioning. As noted in Section C.4.1, actions enter D additively on the caption embedding. Whether perturbed actions reliably render failure is an empirical question the negative probe has to answer for each task; a task where it does not is a task where this venue does not apply.

One model, three tasks. Pooling buys data efficiency but means the three tasks share capacity; we do not currently measure whether per-task models would predict better.

Real-robot interaction is evaluation-only. No RL touches the hardware: the robot is used to collect demonstrations, to record base-policy rollouts, and to evaluate. This is what makes the venue safe and reset-free, but it also means we never observe how the residual would behave under genuine on-robot exploration.

Task	frozen base [95% CI]	ResFit [95% CI]	SHORE-RL [95% CI]	SHORE-base
PICK_PAPER_ROLL	11/50 = 0.22 [0.13, 0.35]	14/50 = 0.28 [0.17, 0.42]	21/50 = 0.42 [0.29, 0.56]	+0.20
BUILD_BLOCK	5/50 = 0.10 [0.04, 0.21]	0/50 = 0.00 [0.00, 0.07]	14/50 = 0.28 [0.17, 0.42]	+0.18
PICK_CUP	6/50 = 0.12 [0.06, 0.24]	1/50 = 0.02 [0.00, 0.10]	16/50 = 0.32 [0.21, 0.46]	+0.20

Table 11: Physical-robot success for the frozen base (step 0), ResFit (step 500k), and SHORE-RL (step 500k). Each entry reports successes out of 50, success rate, and the Wilson 95% interval. The final column is the descriptive SHORE-RL-minus-base difference between marginal rates, not a paired estimate.

Task	Failure statistic	base	ResFit	SHORE-RL
PICK_PAPER_ROLL	total failures	39	36	29
	mode breakdown		[TODO: P1]	
BUILD_BLOCK	total failures	45	50	36
	mode breakdown		[TODO: P1]	
PICK_CUP	total failures	44	49	34
	mode breakdown		[TODO: P1]	

Table 12: Observed failure totals, counts out of 50 trials per cell. Totals are 50 minus the successes in Table 11; they do not identify why a trial failed. **[TODO: P1: define the failure-mode categories per task from the recorded trials, with missed grasps, slips during transport, and misplacements as candidate categories, then allocate the totals. Categories must be fixed before scoring, not after.]**

References

- Ankile, L.; Jiang, Z.; Duan, R.; Shi, G.; Abbeel, P.; and Nagabandi, A. 2026. Residual Off-Policy RL for Finetuning Behavior Cloning Policies. In *IEEE International Conference on Robotics and Automation (ICRA)*.
- Ball, P. J.; Smith, L.; Kostrikov, I.; and Levine, S. 2023. Efficient Online Reinforcement Learning with Offline Data. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, 1577–1594. PMLR.
- Black, K.; Brown, N.; Darpinian, J.; Dhabalia, K.; Driess, D.; Esmail, A.; Equi, M. R.; Finn, C.; Fusai, N.; Galliker, M. Y.; Ghosh, D.; Groom, L.; Hausman, K.; Ichter, B.; Jakubczak, S.; Jones, T.; Ke, L.; LeBlanc, D.; Levine, S.; Li-Bell, A.; Mothukuri, M.; Nair, S.; Pertsch, K.; Ren, A. Z.; Shi, L. X.; Smith, L.; Springenberg, J. T.; Stachowicz, K.; Tanner, J.; Vuong, Q.; Walke, H.; Walling, A.; Wang, H.; Yu, L.; and Zhilinsky, U. 2025a. $\pi_{0.5}$: A Vision-Language-Action Model with Open-World Generalization. In *Proceedings of the 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, 17–40. PMLR.
- Black, K.; Brown, N.; Driess, D.; Esmail, A.; Equi, M. R.; Finn, C.; Fusai, N.; Groom, L.; Hausman, K.; Ichter, B.; Jakubczak, S.; Jones, T.; Ke, L.; Levine, S.; Li-Bell, A.; Mothukuri, M.; Nair, S.; Pertsch, K.; Shi, L. X.; Smith, L.; Tanner, J.; Vuong, Q.; Walling, A.; Wang, H.; and Zhilinsky, U. 2025b. π_0 : A Vision-Language-Action Flow Model for General Robot Control. In *Proceedings of Robotics: Science and Systems*. Los Angeles, California, USA.
- Devlin, S.; and Kudenko, D. 2012. Dynamic Potential-Based Reward Shaping. In *International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 433–440.
- Fujimoto, S.; van Hoof, H.; and Meger, D. 2018. Addressing Function Approximation Error in Actor-Critic Methods. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, 1587–1596. PMLR.
- HaCohen, Y.; Chiprut, N.; Brazowski, B.; Shalem, D.; Moshe, D.; Richardson, E.; Levin, E.; Shiran, G.; Zabari, N.; Gordon, O.; Panet, P.; Weissbuch, S.; Kulikov, V.; Bitterman, Y.; Melumian, Z.; and Bibi, O. 2025. LTX-Video: Realtime Video Latent Diffusion. arXiv:2501.00103.
- Hu, H.; Mirchandani, S.; and Sadigh, D. 2024. Imitation Bootstrapped Reinforcement Learning. In *Proceedings of Robotics: Science and Systems*. Delft, Netherlands.
- Janner, M.; Fu, J.; Zhang, M.; and Levine, S. 2019. When to Trust Your Model: Model-Based Policy Optimization. In *Advances in Neural Information Processing Systems*, volume 32.
- Jiang, Z.; Xie, Y.; Lin, K.; Xu, Z.; Wan, W.; Mandlekar, A.; Fan, L.; and Zhu, Y. 2025. DexMimicGen: Automated Data Generation for Bimanual Dexterous Manipulation via Imitation Learning. In *IEEE International Conference on Robotics and Automation (ICRA)*, 16923–16930.
- Kostrikov, I.; Nair, A.; and Levine, S. 2022. Offline Reinforcement Learning with Implicit Q-Learning. In *International Conference on Learning Representations*.
- Lipman, Y.; Chen, R. T. Q.; Ben-Hamu, H.; Nickel, M.; and Le, M. 2023. Flow Matching for Generative Modeling. In *International Conference on Learning Representations*.
- Liu, B.; Zhu, Y.; Gao, C.; Feng, Y.; Liu, Q.; Zhu, Y.; and Stone, P. 2023. LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning. In *Advances in Neural Information Processing Systems*, volume 36, 44776–44791. Datasets and Benchmarks Track.
- Mandlekar, A.; Nasiriany, S.; Wen, B.; Akinola, I.; Narang, Y.; Fan, L.; Zhu, Y.; and Fox, D. 2023. MimicGen: A Data Generation System for Scalable Robot Learning using Human Demonstrations. In *Proceedings of the 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, 1820–1864. PMLR.
- Ng, A. Y.; Harada, D.; and Russell, S. 1999. Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping. In *International Conference on Machine Learning (ICML)*, 278–287.

- Raffel, C.; Shazeer, N.; Roberts, A.; Lee, K.; Narang, S.; Matena, M.; Zhou, Y.; Li, W.; and Liu, P. J. 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research*, 21(140): 1–67.
- Wagenmaker, A.; Zhang, Y.; Nakamoto, M.; Park, S.; Yagoub, W.; Nagabandi, A.; Gupta, A.; and Levine, S. 2025. Steering Your Diffusion Policy with Latent Space Reinforcement Learning. In *Proceedings of the 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, 258–282. PMLR.
- Yang, J.; Lin, K.; Zhang, W.; Li, J.; Lin, T.; Wu, L.; Zhang, Y.-Q.; Zhao, H.; Luo, P.; Su, Z.; Li, H.; Yue, X.; and Chen, L. 2026. Self-Improving Robot Policy with Compositional World Model. In *Proceedings of Robotics: Science and Systems*.
- Yu, C.; Sima, C.; Jiang, G.; Zhang, H.; Mai, H.; Li, H.; Wang, H.; Chen, J.; Wu, K.; Chen, L.; Zhao, L.; Shi, M.; Luo, P.; Bu, Q.; Peng, S.; Li, T.; and Yuan, Y. 2026. χ_0 : Resource-Aware Robust Manipulation via Taming Distributional Inconsistencies. arXiv:2602.09021.
- Zhao, T. Z.; Kumar, V.; Levine, S.; and Finn, C. 2023. Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware. In *Proceedings of Robotics: Science and Systems*.